



1

Introduction to .NET MAUI

This chapter is all about getting to know **.NET Multi-platform App UI** (**.NET MAUI**) and what to expect from it. .NET MAUI enables you to build native cross-platform mobile and desktop apps for Android, iOS, macOS, and Windows using .NET and C#. This is the only chapter that is purely theoretical; all the others cover hands-on projects. You are not expected to write any code at this point, but instead, simply read through this chapter to develop a high-level understanding of what .NET MAUI is, how .NET MAUI relates to .NET, and how to set up a development machine.

We will start by defining what a native app is and what .NET as a technology brings to the table. After that, we will look at how .NET MAUI fits into the bigger picture and learn when it is appropriate to use the traditional .NET mobile and .NET MAUI apps. We often use the term **traditional .NET mobile app** to describe apps that don't use .NET MAUI, even though .NET MAUI apps are bootstrapped through a traditional .NET mobile app.

In this chapter, we will cover the following topics:

- Defining native applications
- .NET mobile
- Exploring the .NET MAUI framework
- Setting up our development machine
- .NET mobile productivity tooling

Let's get started!

Defining native applications

The term **native application** means different things to different people. For some people, it refers to an app that is developed using the tools specified by the creator of the platform, such as an app developed for iOS with Objective-C or Swift, an Android app developed with Java or Kotlin, or a Windows app developed with C/C++ or the .NET Framework. Others use the term native application to refer to apps that are compiled into machine code that is native to the platform architecture, for example, x86, x64, or ARM. In this book, we will define a native application as one that has a native UI, performance, and API access. The following list explains these three concepts in greater detail:

- **Native UI:** Apps built with .NET MAUI use the standard controls for each platform. This means, for example, that an iOS app built with .NET MAUI will look and behave as an iOS user would expect and an Android app built with .NET MAUI will look and behave as an Android user would expect.
- **Native performance:** Apps built with .NET MAUI are compiled for native performance, meaning that they execute at nearly the same levels as apps built with the tools designed for the platform, that is, Java or Swift, and can use platform-specific hardware acceleration.
- **Native API access:** Native API access means that apps built with .NET MAUI can use everything that the target platforms and devices offer to developers. For example, .NET MAUI applications can use hardware-specific features such as the camera or maps.

.NET mobile

.NET mobile (formerly known as Xamarin) is a set of extensions to .NET that is used to develop native applications for iOS (**.NET for iOS/tvOS/Mac Catalyst**), Android (**.NET for Android**), and macOS (**.NET for macOS**). .NET is the evolution of the .NET Framework, designed for cross-platform development. .NET mobile was introduced in .NET Core 5 as optional workloads. It is technically a binding layer on top of these platforms. Using bindings to platform APIs enables .NET developers to use C# (and F#) to develop native applications with the full capacity of each platform.

The C# APIs we use when we develop apps with .NET mobile match the platform APIs, but they are modified to adhere to conventions used in .NET Core. For example, APIs are often customized to follow .NET naming conventions, and the Android **set** and **get** methods are often replaced by properties. This makes using the APIs easier and more familiar for .NET developers.

Mono (<https://www.mono-project.com>) is an open source implementation of the Microsoft .NET Framework, which is based on the **European Computer Manufacturers Association (ECMA)** standards for C# and the **Common Language Runtime (CLR)**. Mono was created to bring the .NET Framework to platforms other than Windows. It is part of the .NET Foundation (<http://www.dotnetfoundation.org>), an independent organization that supports open development and collaboration involving the .NET ecosystem. Since .NET 5, Mono is now a supported runtime for applications built on .NET. No separate installer is needed to use Mono with .NET; it is included in the installer for .NET. The Mono runtime is used for iOS, tvOS, Mac Catalyst, and Android applications, while the .NET Core CLR is used for all other supported platforms.

With a combination of the .NET mobile platforms, .NET, and Mono, we can use both the platform-specific APIs and the platform-independent parts of .NET, including namespaces such as **System**, **System.Linq**, **System.IO**, **System.Net**, and **System.Threading.Tasks**.

There are several reasons for using .NET mobile for mobile app development, which we will cover in the following sections.

Code sharing

If we use one common programming language for multiple mobile platforms (and even server platforms), then we can share a lot of code between our target platforms, as illustrated in the following diagram. All code that isn't related to the target platform can be shared with other .NET platforms. Code that is typically shared in this way includes business logic, network calls, and data models:

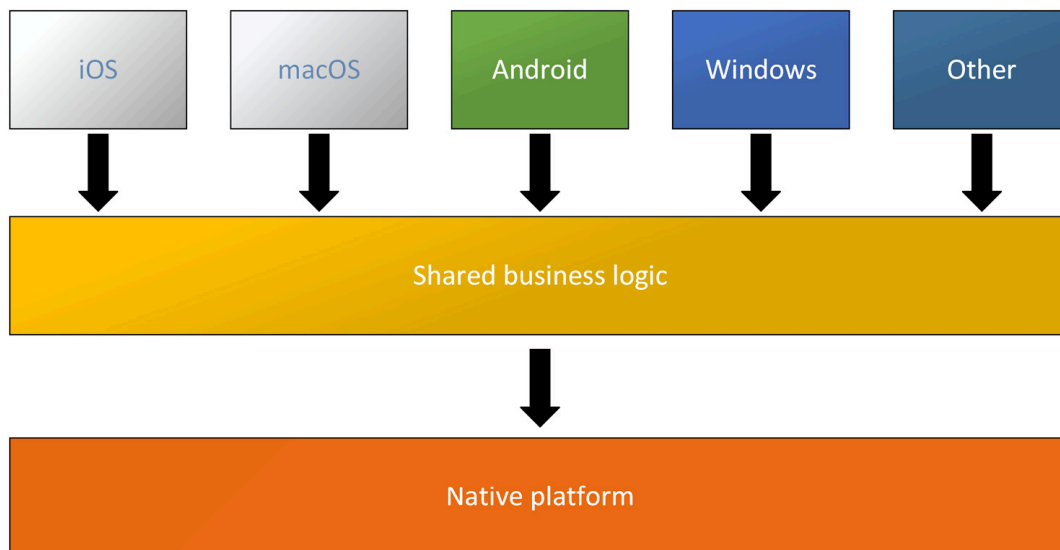


Figure 1.1 – .NET MAUI code sharing

There is also a large community based around the .NET platforms, providing a different form of code sharing. There is a wide range of third-party libraries and components that can be downloaded from NuGet (<https://nuget.org>) that can provide you with additional features or capabilities that work across all supported .NET MAUI platforms. For example, you can find NuGet packages that provide databases, graphs, or barcode reading to include in your apps.

Code sharing across platforms leads to shorter development times. It also produces apps of a higher quality because, for example, we only need to write the code for business logic once. There is a lower risk of

bugs, and it is also able to guarantee that a calculation returns the same result, regardless of what platform our users use.

Use of existing knowledge

For .NET developers who want to start building native mobile apps, it is easier to just learn the APIs for the new platforms than it is to learn programming languages and APIs for both old and new platforms.

Similarly, organizations that want to build native mobile apps can use existing developers with their knowledge of .NET to develop apps. Because there are more .NET developers than Objective-C and Swift developers, it's easier to find new developers for mobile app development projects.

.NET mobile platforms

The different .NET mobile platforms available are .NET for iOS/tvOS/Mac Catalyst, .NET for Android, and .NET for macOS. In this section, we will take a look at each of them.

.NET for iOS/tvOS/Mac Catalyst

.NET for iOS/tvOS/Mac Catalyst is used to build apps for iOS, tvOS, or Mac Catalyst, respectively, with .NET and contains the bindings to the iOS APIs mentioned previously. .NET for iOS/tvOS/Mac Catalyst uses **ahead-of-time (AOT)** compilation to compile the C# code into the **Advanced RISC Machine (ARM)** assembly language. The Mono runtime runs alongside the Objective-C runtime. Code that uses .NET namespaces, such as **System.Linq** or **System.Net**, is executed by the Mono runtime, while code that uses iOS-specific namespaces is executed by the Objective-C runtime. Both the Mono runtime and the Objective-C runtime run on top of the **X is Not Unix (XNU)** Unix-like kernel (<https://github.com/apple/darwin-xnu>), which was developed

by Apple. The following diagram shows an overview of the iOS architecture:

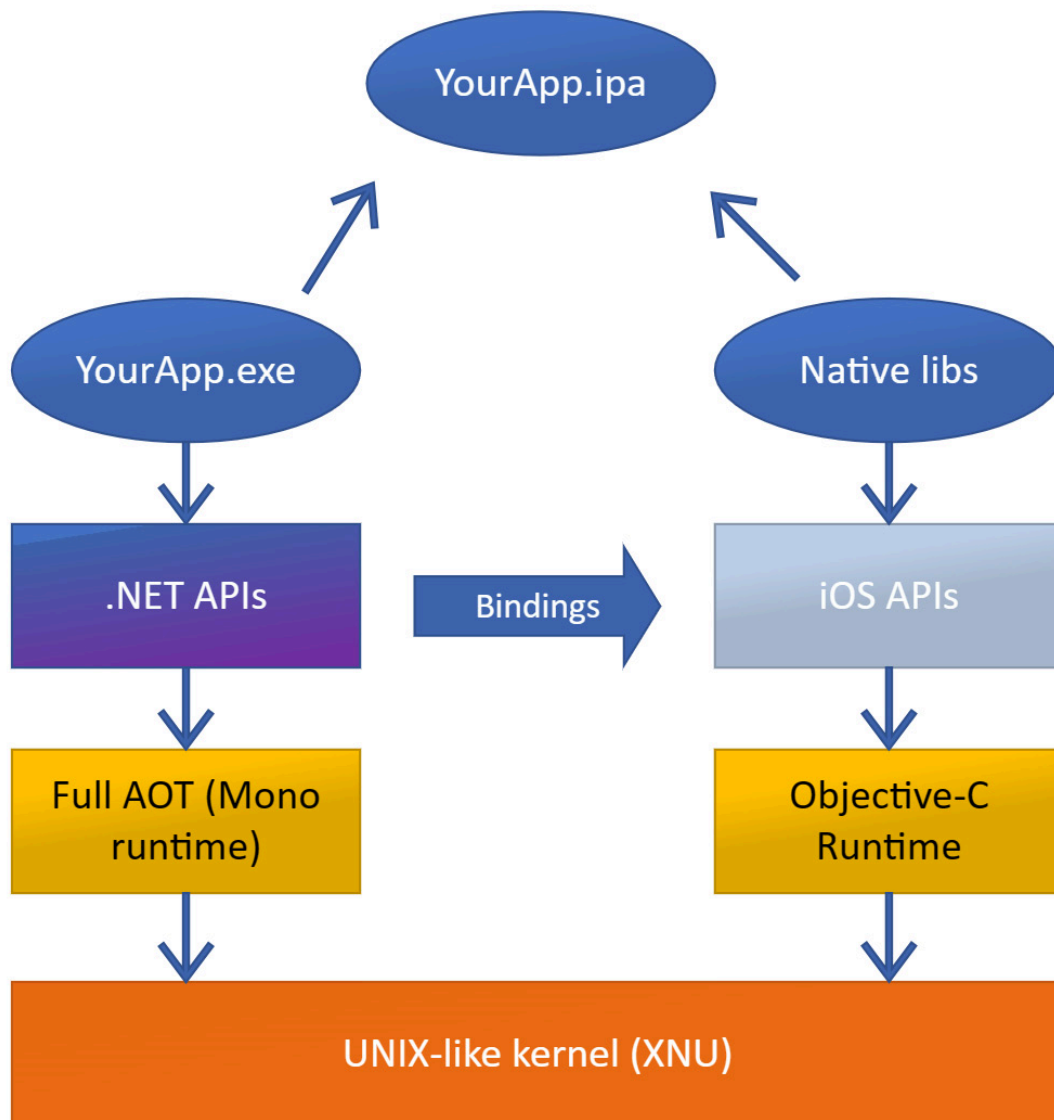


Figure 1.2 – .NET for iOS

.NET for macOS

.NET for macOS is used to build apps for macOS with .NET and contains the bindings to the macOS APIs. .NET for macOS has the same architecture as .NET for iOS—the only difference is, .NET for macOS apps are **just-in-time (JIT)**-compiled, unlike .NET for iOS apps, which are AOT-compiled. This is shown in the following diagram:

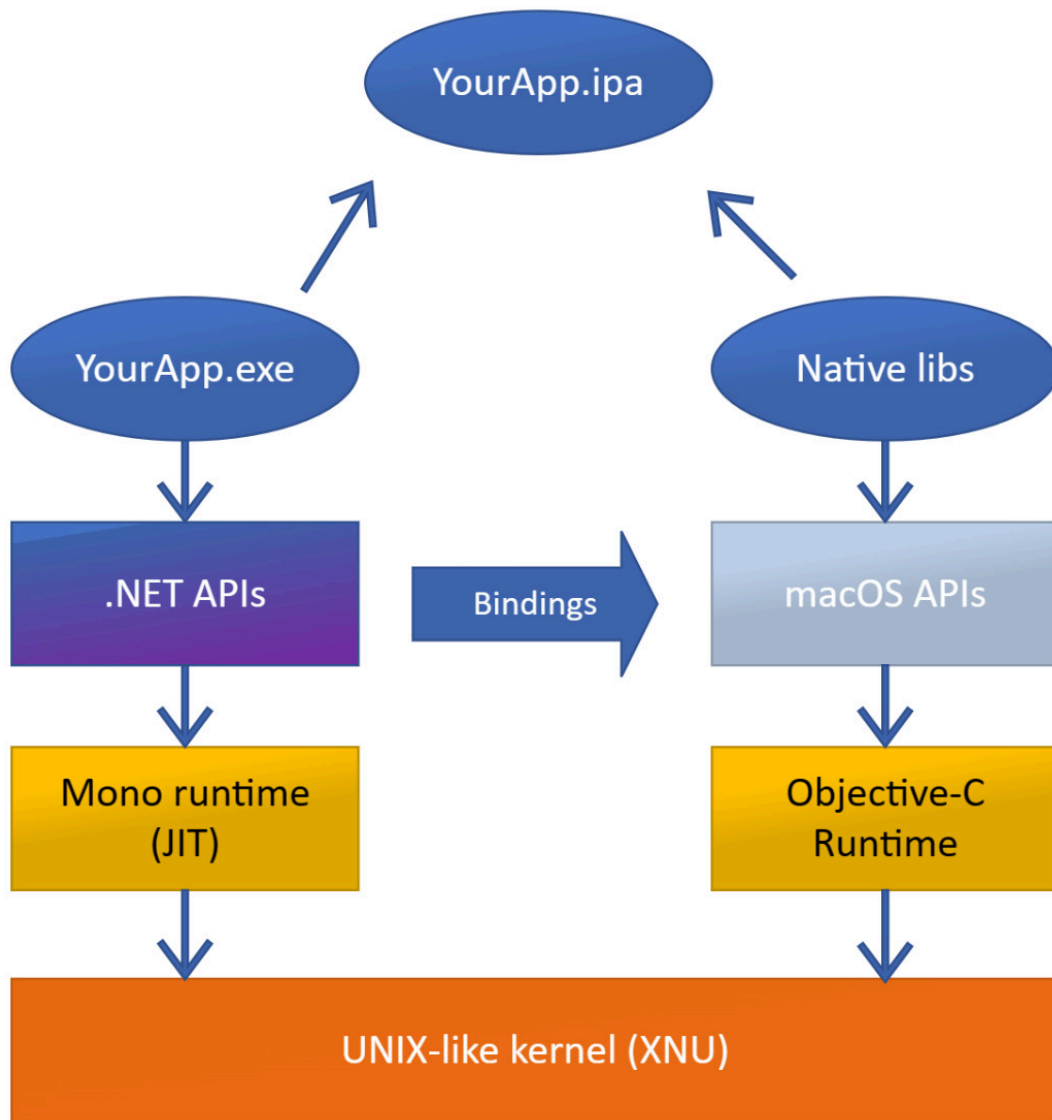


Figure 1.3 – .NET for macOS

.NET for Android

.NET for Android is used to build apps for Android with .NET and contains bindings to the Android APIs. The Mono runtime and the **Android Runtime (ART)** run side by side on top of a Linux kernel. .NET for Android apps could either be JIT-compiled or AOT-compiled, but to AOT-compile them, we need to use Visual Studio Enterprise.

Communication between the Mono runtime and ART occurs via a **Java Native Interface (JNI)** bridge. There are two types of JNI bridges—**Managed Callable Wrapper (MCW)** and **Android Callable Wrapper**

(**ACW**). An MCW is used when code needs to run in ART and an ACW is used when ART needs to run code in the Mono runtime, as shown:

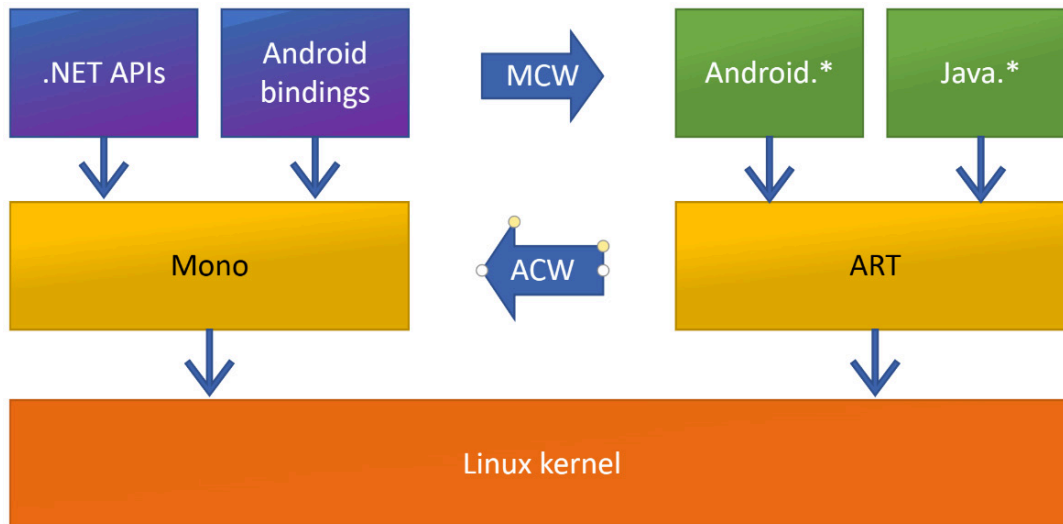


Figure 1.4 – .NET for Android

.NET FOR TIZEN

.NET MAUI has additional support for the Tizen platform from Samsung. Samsung provides the binding layer and runtime to allow .NET MAUI to run on the Tizen platform. To learn more about how to install and develop for the Tizen platform, visit <https://github.com/Samsung/Tizen.NET> in your browser.

Now that we understand what .NET mobile is and how each platform works, we can explore .NET MAUI in detail.

Exploring the .NET MAUI framework

.NET MAUI is a cross-platform framework that is built on top of .NET mobile (for iOS and Android) and the **Windows UI (WinUI)** library. .NET MAUI allows developers to create a UI for iOS, Android, and WinUI in XAML. .NET MAUI improves on Xamarin.Forms by placing all platform-specific functionality in the same project as cross-plat-

form functionality, making it easier to find and edit your code. .NET MAUI also includes all of what used to be in Xamarin.Essentials, which provides cross-platform capabilities, such as permissions, location, photos and camera, contacts, and maps, and leverages that cross-platform functionality with one shared code base, as illustrated in the following diagram:

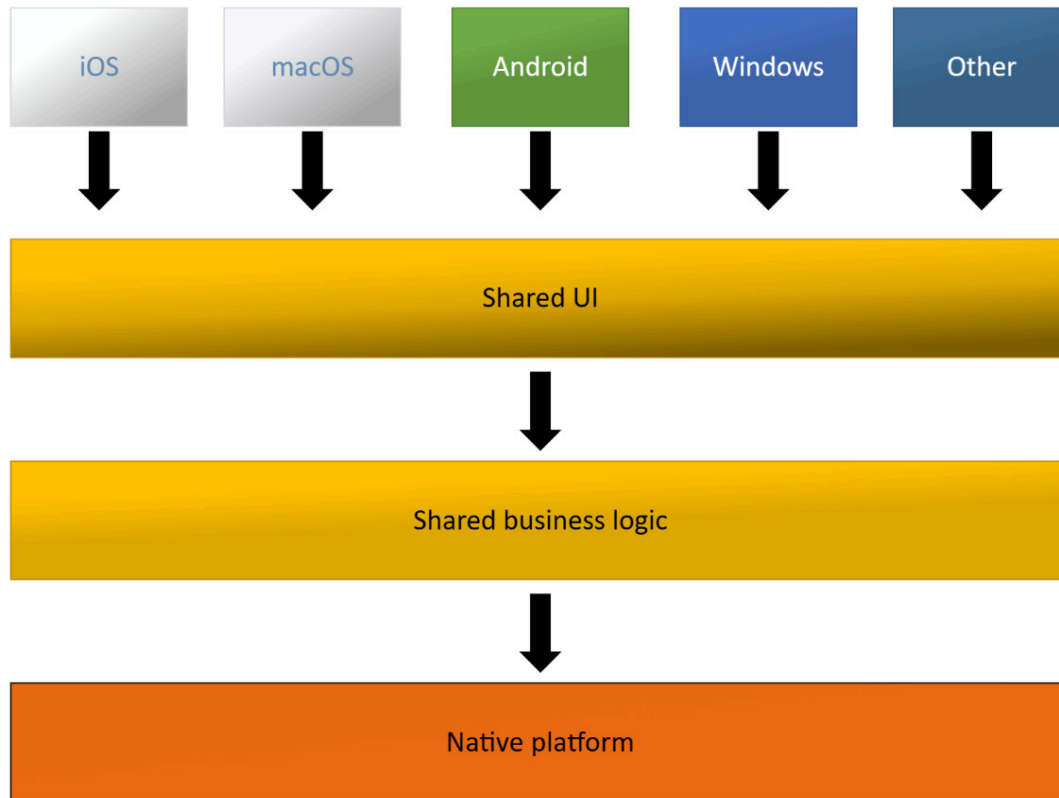


Figure 1.5 – .NET MAUI architecture

If we build an app with .NET MAUI, we can use XAML, C#, or a combination of both to create the UI.

The architecture of .NET MAUI

.NET MAUI is an abstraction layer on top of each platform. .NET MAUI has a shared layer that is used by all platforms, as well as a platform-specific layer. The platform-specific layer contains **handlers**. A handler is a class that maps a .NET MAUI control to a platform-specific native control. Each .NET MAUI control has a platform-specific handler.

The following diagram illustrates how the entry control in .NET MAUI is mapped to the correct native control for each platform. The entry control is mapped to a **UITextField** control from the **UIKit** namespace when the shared .NET MAUI code is used in an iOS app. On Android, the entry control is mapped to an **EditText** control from the **AndroidX.AppCompat.Widget** namespace. Finally, for Windows, .NET MAUI **Entry** handlers map to **TextBox** from the **Microsoft.UI.Xaml.Controls** namespace.

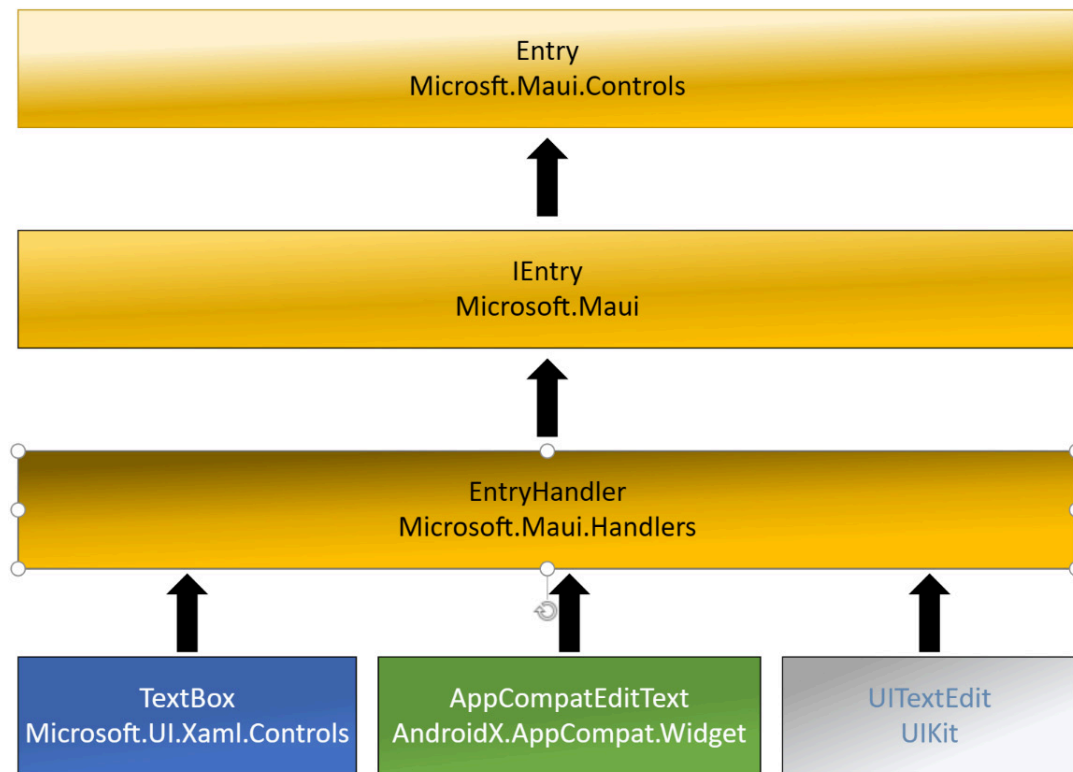


Figure 1.6 – .NET MAUI control architecture

With a firm grasp of .NET MAUI architecture and .NET mobile platforms, it is time to explore how to create UIs in .NET MAUI.

Defining a UI using XAML

The most common way to declare our UI in .NET MAUI is by defining it in a XAML document. It is also possible to create the GUI in C#, since XAML is a markup language for instantiating objects. We could, in theory, use XAML to create any type of object, provided it has a parame-

terless constructor. A XAML document is an **Extensible Markup Language (XML)** document with a specific schema.

Over the next few sections, we are going to learn about a few controls in .NET MAUI to get us started. Then, we will compare different ways that you can construct the UI using .NET MAUI.

Defining a Label control

As a simple example, let's look at the following snippet of a XAML document:

```
<Label Text="Hello World!" />
```

When the XAML parser encounters this snippet, it creates an instance of a **Label** object and then sets the properties of the object that correspond to the attributes in the XAML. This means that if we set a **Text** property in XAML, it sets the **Text** property on the instance of the **Label** object that is created. The XAML in the preceding example has the same effect as the following:

```
var obj = new Label()  
{  
    Text = "Hello World!"  
};
```

XAML exists to make it easier to view the object hierarchy that we need to create in order to make a GUI. An object model for a GUI is also hierarchical by design, so XAML supports adding child objects. We can simply add them as child nodes, as follows:

```
<StackLayout>  
    <Label Text="Hello World" />
```

```
<Entry Text="Ducks are us" />
</StackLayout>
```

StackLayout is a container control that organizes the children vertically or horizontally within a container. Vertical organization is the default value and is used unless we specify otherwise. There are other containers, such as **Grid** and **FlexLayout**.

These will be used in many of the projects in the following chapters.

Creating a page in XAML

A single control is no use unless it has a container that hosts it. Let's see what an entire page would look like. A fully valid **ContentPage** object defined in XAML is an XML document. This means that we must start with an XML declaration. After that, we must have one—and only one—root node, as shown:

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml" x:Class="MyApp.
MainPage">
    <StackLayout>
        <Label Text="Hello world!" />
    </StackLayout>
</ContentPage>
```

In the preceding example, we defined a **ContentPage** object that translates into a single view on each platform. In order to make it valid XAML, we need to specify a default namespace (`xmlns="http://schemas.microsoft.com/dotnet/2021/maui"`) and then add the `x` namespace (`xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"`).

The default namespace lets us create objects without prefixing them, such as the **StackLayout** object. The `x` namespace lets us access prop-

erties such as **x:Class**, which tells the XAML parser which class to instantiate to control the page when the **ContentPage** object is created.

A **ContentPage** object can have only one child. In this case, it's a **StackLayout** control. Unless we specify otherwise, the default layout orientation is vertical. A **StackLayout** object can, therefore, have multiple children. Later in the book, we will touch on more advanced layout controls, such as the **Grid** and **FlexLayout** controls.

As the first child of **StackLayout**, we will create a **Label** control.

Creating a page in C#

For clarity, the following code shows you how the previous example would look in C#:

```
public class MainPage : ContentPage
{
}
```

MainPage is a class that inherits from .NET MAUI's **ContentPage**. This class is automatically generated for us if we create a XAML page, but if we just use code, we will need to define it ourselves.

Let's create the same control hierarchy as the XAML page we defined earlier using the following code:

```
var page = new MainPage();
var stacklayout = new StackLayout();
stacklayout.Children.Add(
    new Label()
    {
        Text = "Welcome to .NET MAUI"
```

```
});  
page.Content = stacklayout;
```

The first statement creates a **page** object. We could, in theory, create a new **ContentPage** page directly, but this would prohibit us from writing any code behind it. For this reason, it's good practice to subclass each page that we plan to create.

The block following this first statement creates the **StackLayout** control, which contains the **Label** control that is added to the **Children** collection.

Finally, we need to assign **StackLayout** to the **Content** property of the page.

Since XAML is a markup language that mainly instantiates objects for us, we can see how easy it is to replicate that in C#. Next, we will take a look at some extensions that make developing your UI in C# a little better.

Using the .NET MAUI Markup Community Toolkit

The Community Toolkit organization on GitHub has a project to add **Fluent** extensions to MAUI for creating the UI in C#. The project **.NET MAUI Markup Community Toolkit**

(<https://github.com/CommunityToolkit/Maui.Markup>), or **MAUI.Markup** for short, is described on the website as follows:

The .NET MAUI Markup Community Toolkit is a collection of Fluent C# Extension Methods that allows developers to continue architecting their apps using MVVM, Bindings, Resource Dictionaries, etc., without the need for XAML.

Using MAUI Markup to create the same page we did in the previous two sections would look something like the following:

```
public class MainPage: ContentPage
{
    public MainPage()
    {
        Build();
    }
    public void Build() {
        Content = new StackLayout()
        {
            Children =
            {
                new Label()
                {
                    Text = "Welcome to .NET MAUI"
                }
            }
        };
    }
}
```

For more information on how to use MAUI Markup in your applications, visit <https://github.com/CommunityToolkit/Maui.Markup> using your favorite web browser.

So, what is better for creating our UI, XAML or C#?

XAML or C#?

Generally, using XAML provides a much better overview, since the page is a hierarchical structure of objects, and XAML is a very nice way of defining that structure. In code, the structure is flipped around as we need to define the innermost object first, making it harder to read the structure of our page. This was demonstrated in the *Creating a page in XAML* section of this chapter. Having said that, it is generally a matter of preference as to how we decide to define the GUI. This book will use XAML rather than C# in the projects to come.

Now that we've explored how to create our pages using .NET MAUI, it is time to review how .NET MAUI and .NET mobile compare.

.NET MAUI versus traditional .NET mobile

While this book is about .NET MAUI, we will also highlight the differences between using traditional .NET mobile and .NET MAUI.

Traditional .NET mobile is used when developing UIs that use the iOS or Android **software development kit (SDK)** without any means of abstraction. For example, we can create an iOS app that defines its UI in a storyboard or in the code directly. This code would not be reusable for other platforms, such as Android. Apps built using this approach can still share non-platform-specific code by simply referencing a .NET standard library. This relationship is shown in the following diagram:

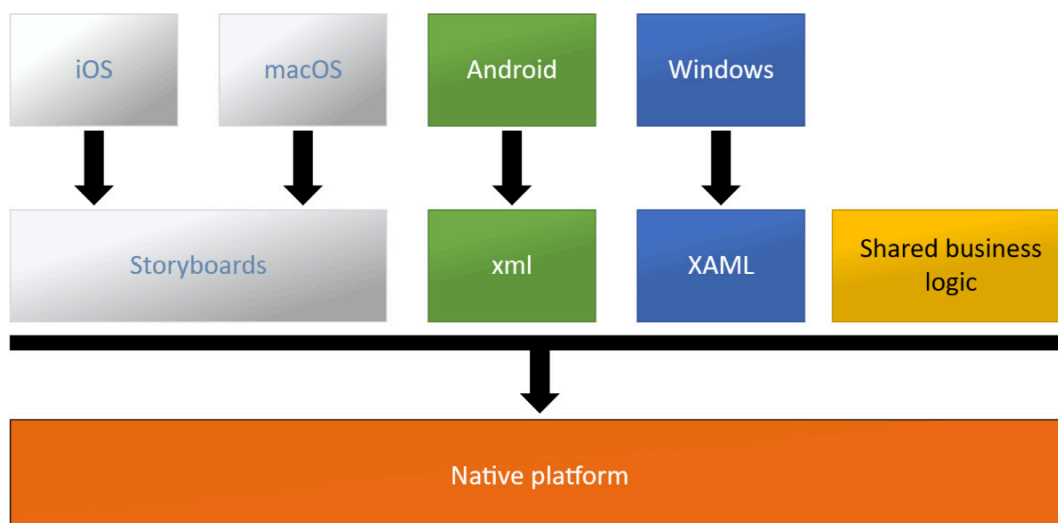


Figure 1.7 – Traditional .NET UI

.NET MAUI, on the other hand, is an abstraction of the GUI, which allows us to define UIs in a platform-agnostic way. It still builds on top of .NET for iOS, .NET for Android, and all the other supported platforms. The .NET MAUI app is created as a .NET standard library where the shared source files and platform-specific source files are all built within the same project for the platform we are currently building for. This relationship is shown in the following diagram:

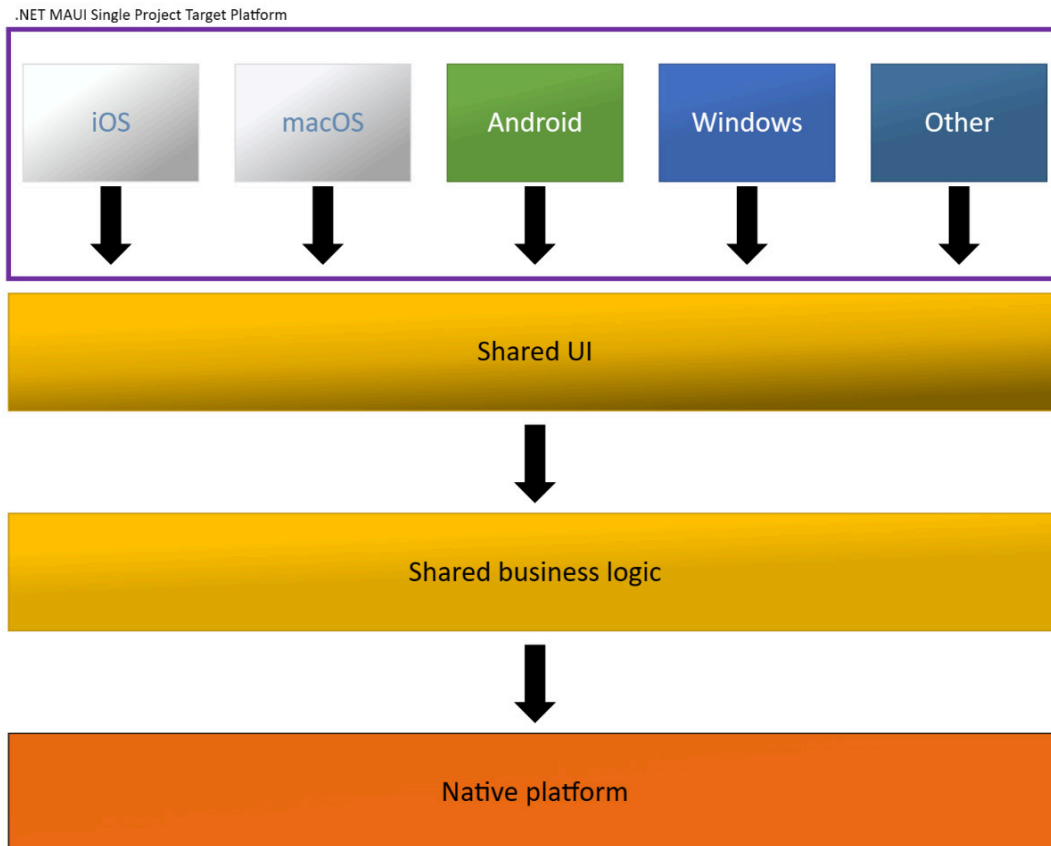


Figure 1.8 – .NET MAUI UI with a single project

Having said that, .NET MAUI cannot exist without traditional .NET mobile since it's bootstrapped through an app for each platform. This gives us the ability to extend .NET MAUI on each platform using custom renderers and platform-specific code that can be exposed to our shared code base through interfaces. We'll look at these concepts in more detail later in this chapter.

When to use .NET MAUI

We can use .NET MAUI in most cases and for most types of apps. If we need to use controls that are not available in .NET MAUI, we can always use the platform-specific APIs. There are, however, cases where .NET MAUI is not useful. The most common situation where we might want to avoid using .NET MAUI is if we build an app that should look very different across our different target platforms.

Enough theory for now; let's get our development machines ready to develop using .NET MAUI.

Setting up our development machine

Developing an app for multiple platforms imposes higher demands on our development machine. One reason for this is that we often want to run one or multiple simulators or emulators on our development machine. Different platforms also have different requirements for what is needed to begin development. Regardless of whether we use macOS or Windows, Visual Studio will be our **integrated development environment (IDE)**. There are several versions of Visual Studio, including the free community edition. Go to

<https://visualstudio.microsoft.com/> to compare the available versions, and select the version that is right for you; .NET MAUI is included in all versions of Visual Studio for Windows and macOS. The following list is a summary of what we need to begin development for each platform:

- **iOS:** To develop an app for iOS, we need a **Macintosh (Mac)** device. This could either be the machine that we are developing on or a machine on our network, if we are using one. The reason we need to connect to a Mac is we need Xcode to build the app package. Xcode also provides various simulators to run and debug your app. It is possible to do some iOS development on Windows without a

connected Mac; you can read more about this in the *Xamarin Hot Restart* section of this chapter.

- **Android:** Android apps can be developed on either macOS or Windows. Everything we need, including SDKs and simulators, is installed with Visual Studio.
- **WinUI:** WinUI apps can only be developed in Visual Studio on a Windows machine.

We'll start with setting up a Mac first, and then later cover Windows. If you do not own a Mac, you can skip this section and head straight to the *Setting up a Windows machine* section.

Setting up a Mac

There are two main tools that are required to develop apps for iOS and Android with .NET mobile on a Mac. These are Visual Studio for Mac (if we are only developing Android apps, this is the only tool we need) and Xcode. In the following sections, we will look at how to set up a Mac for app development.

Installing Xcode

Before we install Visual Studio, we need to download and install Xcode. Xcode is the official development IDE from Apple and contains all the tools available for iOS development, including SDKs for iOS, macOS, Mac Catalyst, and tvOS.

We can download Xcode from the Apple developer portal (<https://developer.apple.com>) or the Apple App Store. I recommend that you download it from the App Store because this guarantees you have the latest stable version. The only reason to download Xcode from the developer portal is if you want to use a prerelease version of Xcode to develop it for a prerelease of iOS.

When using prerelease versions of macOS and its accompanying version of Xcode, it is possible that .NET for iOS/tvOS/Mac Catalyst/macOS has not been updated to work with the latest Xcode changes. It is recommended to check the compatibility before installing the latest Xcode to ensure a working environment.

After the first installation, and after each update of Xcode, it is important that you open it. Xcode often needs to install additional components after an installation or an update. We also need to open Xcode to accept the license agreement with Apple.

Installing Visual Studio

To install Visual Studio, we first need to download it from <https://visualstudio.microsoft.com>.

VISUAL STUDIO FOR MAC DEPRECATION

On August 31, 2023, Microsoft announced the deprecation of Visual Studio for Mac in accordance with their Modern Lifecycle Policy, which will mean the end of support on August 31, 2024. Visual Studio for Mac will remain supported until that date. If you have a Visual Studio subscription, you can always download the latest version of Visual Studio for Mac from my.visualstudio.com. Microsoft has released the C# Dev Kit and .NET MAUI Dev Kit extensions for Visual Studio Code. The extensions work on Windows, macOS, and Linux. You can use Visual Studio Code and these extensions to complete the projects in this book, although the instructions pertaining to the UI for Visual Studio for Mac will not match Visual Studio Code. To learn more, visit <https://learn.microsoft.com/en-us/visualstudio/mac/what-happened-to-vs-for-mac?view=vsmac-2022>.

When we start the Visual Studio installer via the file we downloaded, it will start to check what we already have installed on our machine.

When the check is finished, we can select which platforms and tools we would like to install.

Once we have selected the platforms that we want to install, Visual Studio downloads and installs everything that we need to get started with app development using .NET mobile, as shown:

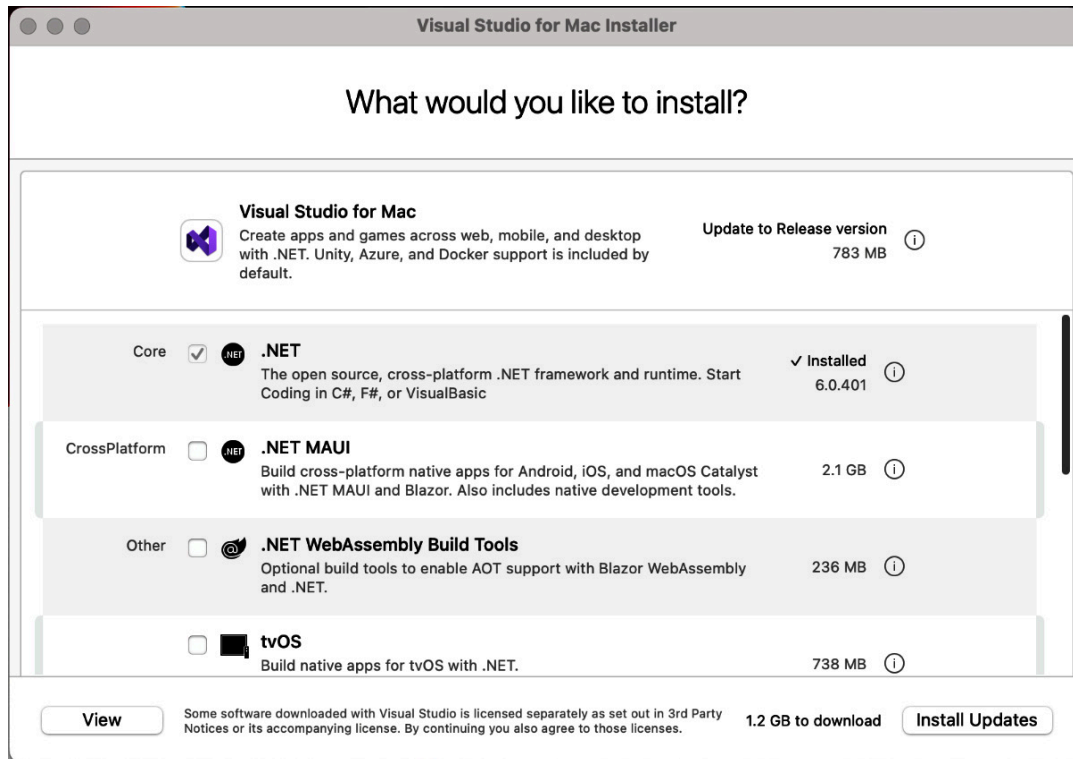


Figure 1.9 – Visual Studio for Mac installer

Configuring the Android Emulator

Visual Studio uses the Android emulators provided by Google. They are installed and configured through SDK Manager.

SPECIAL NOTE FOR INTEL-BASED MAC DEVICE

*If we want our emulator to be fast, then we need to ensure that it is hardware-accelerated. To hardware-accelerate the Android emulator, we need to install the Intel **Hardware Accelerated Execution Manager (HAXM)**, which can be downloaded from*

<https://software.intel.com/en-us/articles/intel-hardware-accelerated-execution-manager-intel-haxm>.

The next step is to create the Android emulator. First, we need to ensure that the Android emulator and the Android OS images are installed. To do this, take the following steps:

1. Go to the **Tools** tab to install the Android emulator:

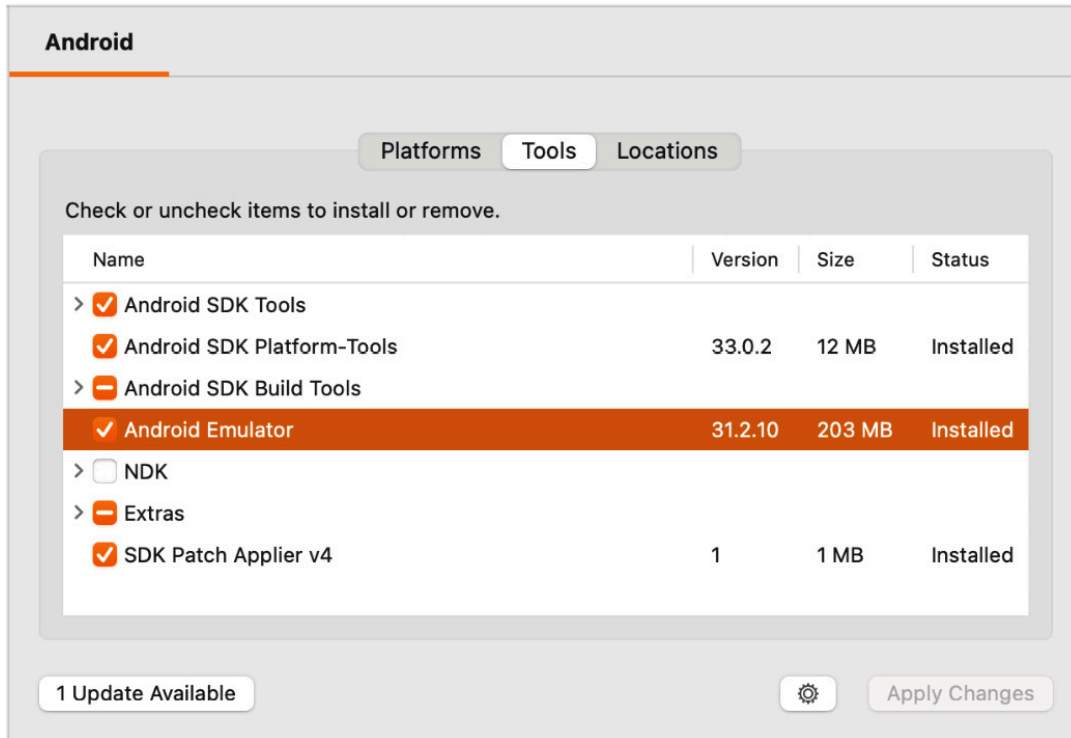


Figure 1.10 – Installing Android Emulator in Visual Studio for Mac

2. We also need to install one or multiple images to use with the emulator. We can install multiple images if, for example, we want to run our app on different versions of Android. We can select emulators with Google Play (as in the following screenshot) so that we can use Google Play services in our app, even when we are running it in an emulator. This is required if, for example, we want to use Google Maps in our app:

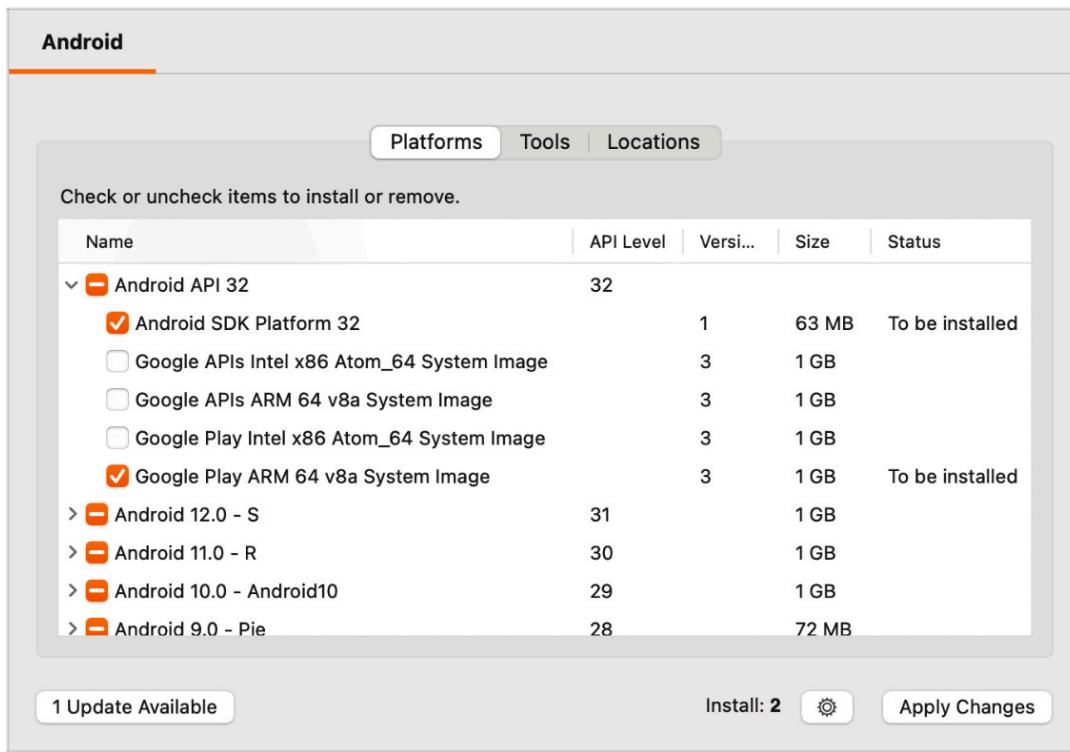


Figure 1.11 – Installing emulator images in Visual Studio for Mac

INTEL VERSUS APPLE M1

*If you have an Apple Mac that uses the M1 chipset, then you should use emulator images that have **ARM 64** in the name; otherwise, if you are using older Mac devices with the Intel chipset, then use images with **Intel x86** in the name.*

- Then, to create and configure an emulator, go to **Device Manager** in the **Tools** menu in Visual Studio. From **Android Device Manager**, we can start an emulator if we already have one created, or we can create new emulators, as shown:



Figure 1.12 – Android Device Manager in Visual Studio for Mac

4. If we click on the **New Device** button, we can create a new emulator with the specifications that we need. The easiest way to create a new emulator here is to select a base device that matches our needs. These base devices are preconfigured, which is often enough. However, it is also possible to edit the properties of the device so that we have an emulator that matches our specific needs. The processor dropdown will be preselected with the correct architecture of your device. If you change this, for example, from ARM to x86 or x86 to ARM, then the emulator will be slower; always try to use the architecture that matches your device.

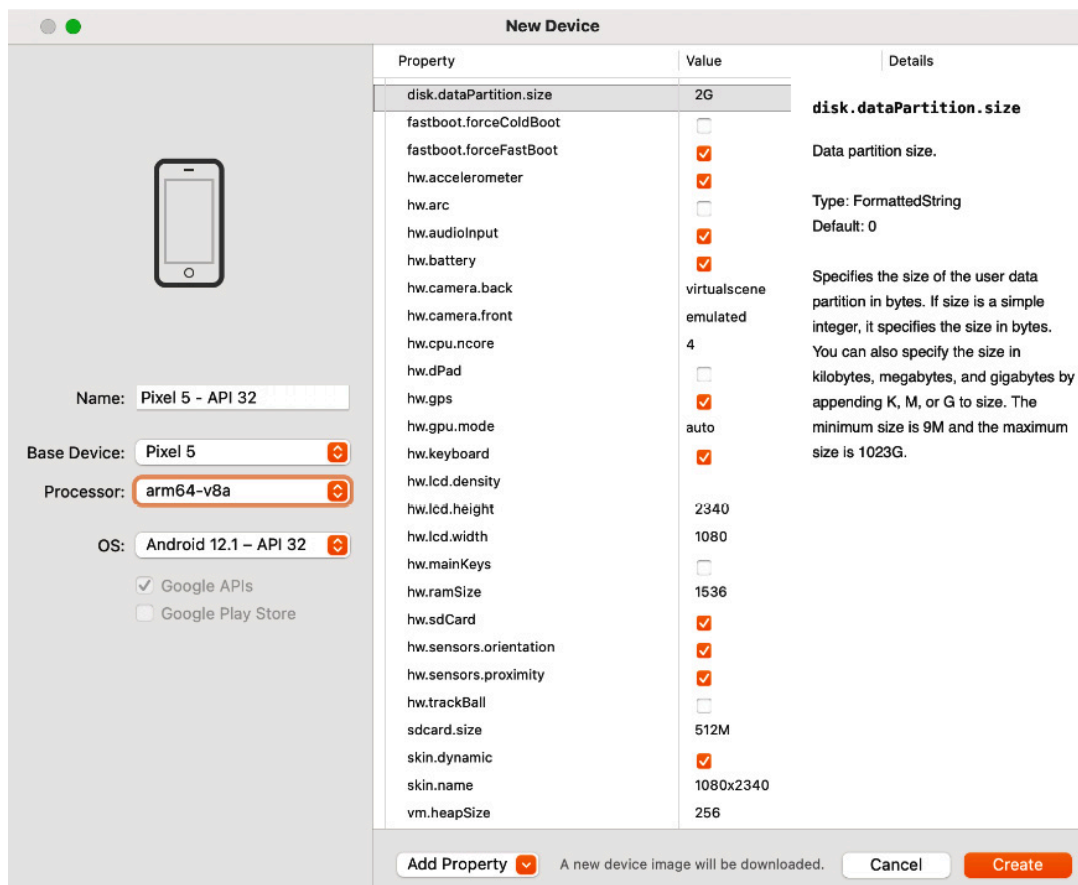


Figure 1.13 – Creating a new Android device

If you only have a Mac, then you are done and can skip to the *.NET mobile productivity tooling* section. If you have a Windows device, then the next section, *Setting up a Windows machine*, is for you.

Setting up a Windows machine

We can use either a virtual or physical Windows machine for development with .NET mobile. We can, for example, run a virtual Windows machine on our Mac. The only tool we need for app development on our Windows machine is Visual Studio.

Installing .NET mobile for Visual Studio 2022 or later

If we already have Visual Studio 2022 or later installed, we first need to open **Visual Studio Installer**; otherwise, we need to go to <https://visualstudio.microsoft.com> to download the installation files. Under the banner in the **Meet the Visual Studio Family** section, you can find the links to download Visual Studio 2022 for Windows or Visual Studio 2022 for Mac.

Before the installation starts, we need to select which workloads we want to install.

For .NET MAUI development, we need to install **.NET Multi-platform App UI development**. Select the **ASP.NET and web development** workload to be able to develop MAUI/Blazor hybrid apps.

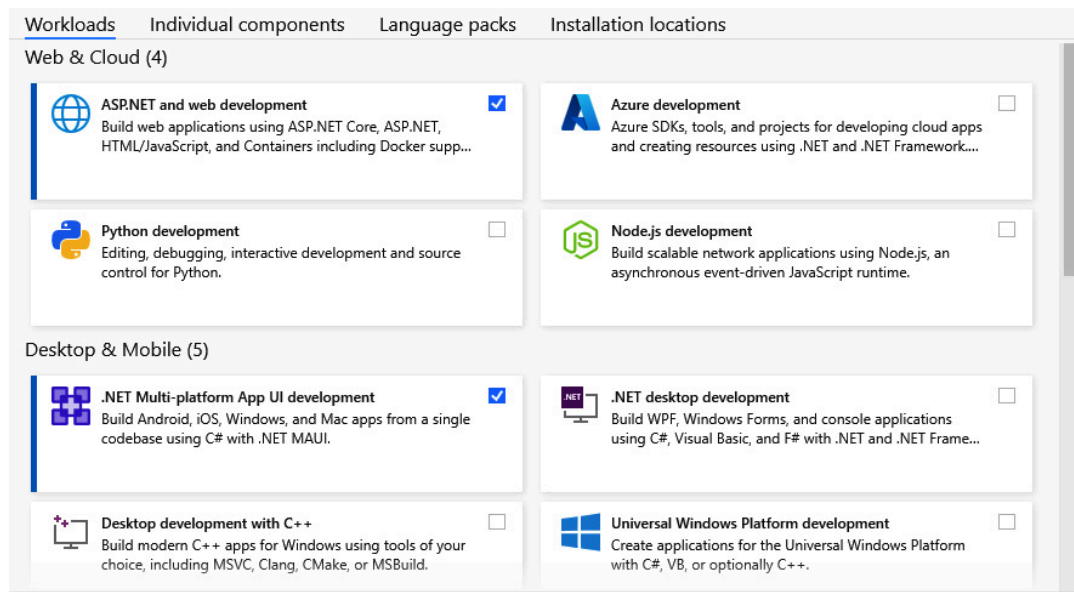


Figure 1.14 – Visual Studio 2022 installer

Hyper-V is the default hardware acceleration method when using .NET MAUI. If you want to use Intel HAXM, you will need to check the checkbox for Intel HAXM in the **Individual components** tab, as in the following screenshot:

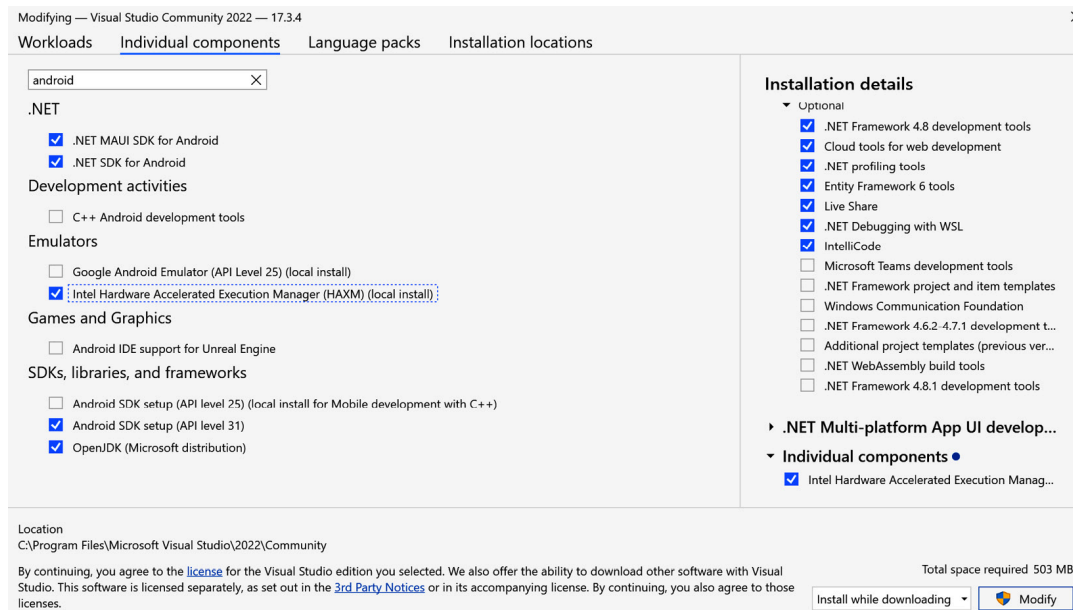


Figure 1.15 – Adding Intel HAXM

When we first start Visual Studio, we will be asked whether we want to sign in. It is not necessary for us to sign in unless we want to use Visual Studio Professional or Enterprise, in which case we will need to sign in so that our license can be verified.

Now that Visual Studio is installed, we can finish the configuration needed to run and debug apps for iOS and Android.

Pairing Visual Studio with a Mac

If we want to run, debug, and compile our iOS app from a Windows development machine, then we need to connect it to a Mac. We can set up our Mac manually, as described earlier in this chapter, or we can use **Automatic Mac Provisioning** from within Visual Studio. This installs Mono and .NET for iOS on the Mac that we are connecting to. It will not install the Visual Studio IDE, but this isn't necessary if we just

want to use it as a build machine. We do, however, need to install Xcode manually.

To connect to the Mac from Visual Studio, use the **Pair to Mac** button in the toolbar (as in the following screenshot), or, in the top menu, go to **Tools | iOS | Pair to Mac**:

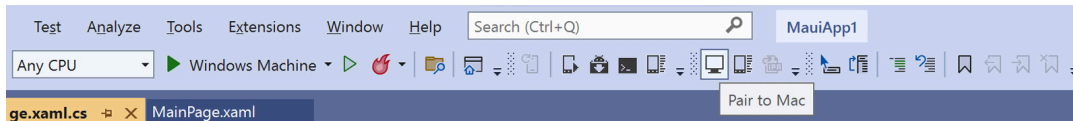


Figure 1.16 – Pair to Mac button

If this is the first time you have attempted to pair to a Mac, Visual Studio will open a wizard that will guide you through the steps you need to take on your Mac to enable Visual Studio to connect.

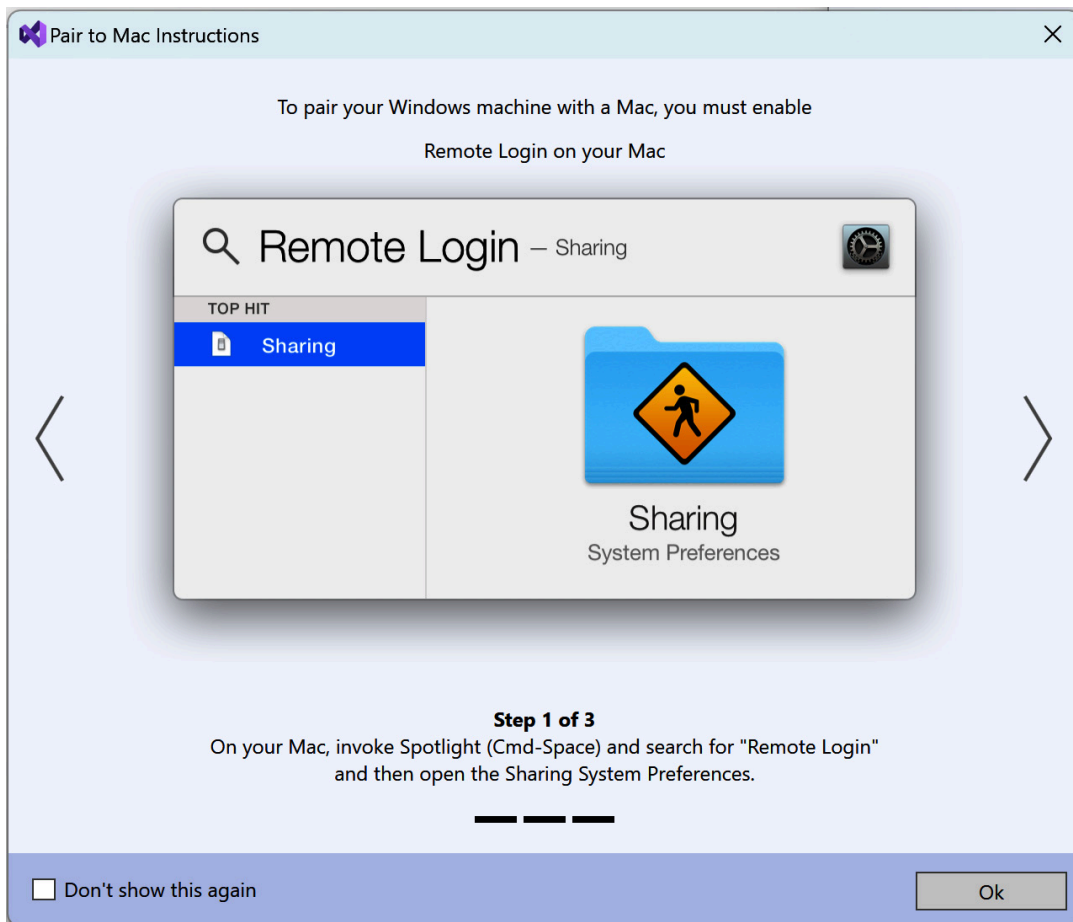


Figure 1.17 – Pair to Mac wizard

To be able to connect to a Mac—either manually or using **Automatic Mac Provisioning**—we need to be able to access the Mac via our network, and we need to enable **Remote Login** on the Mac.

To do this, go to **Settings | Sharing** and select the checkbox for **Remote Login**. To the left of the window, we can select which users are allowed to connect with **Remote Login**, as shown:

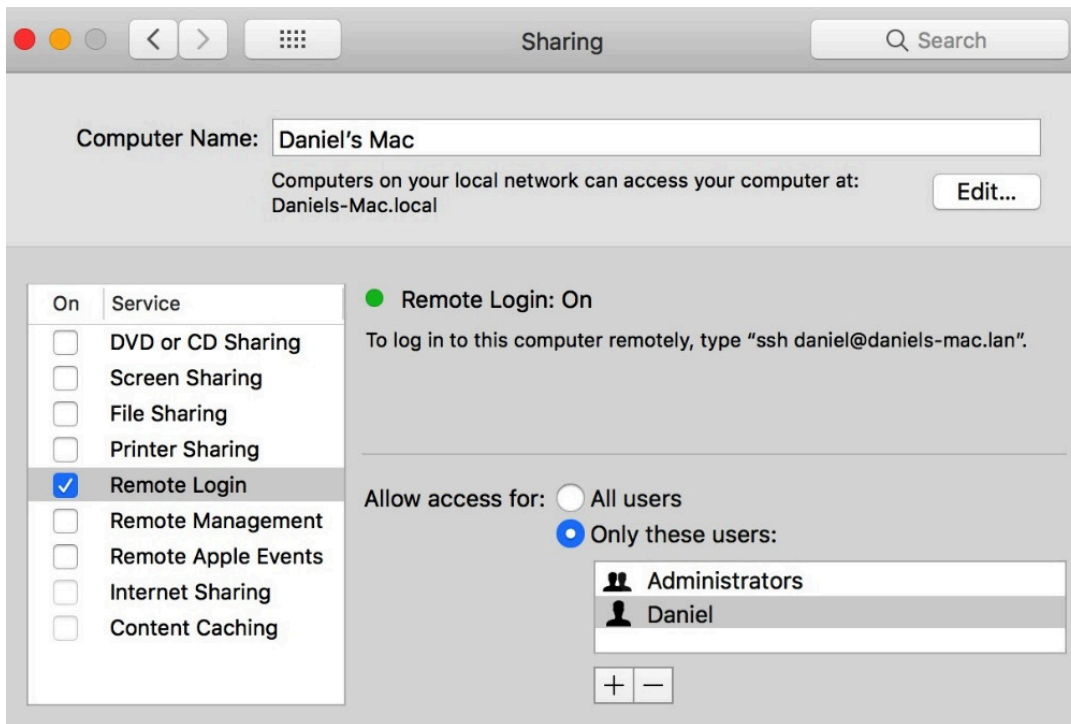


Figure 1.18 – Enabling Remote Login on macOS

A dialog box will appear showing all the Macs that can be found on the network. If your Mac doesn't appear in the list of available Macs, you can use the **Add Mac...** button in the bottom-left corner of the window to enter an IP address, as shown:

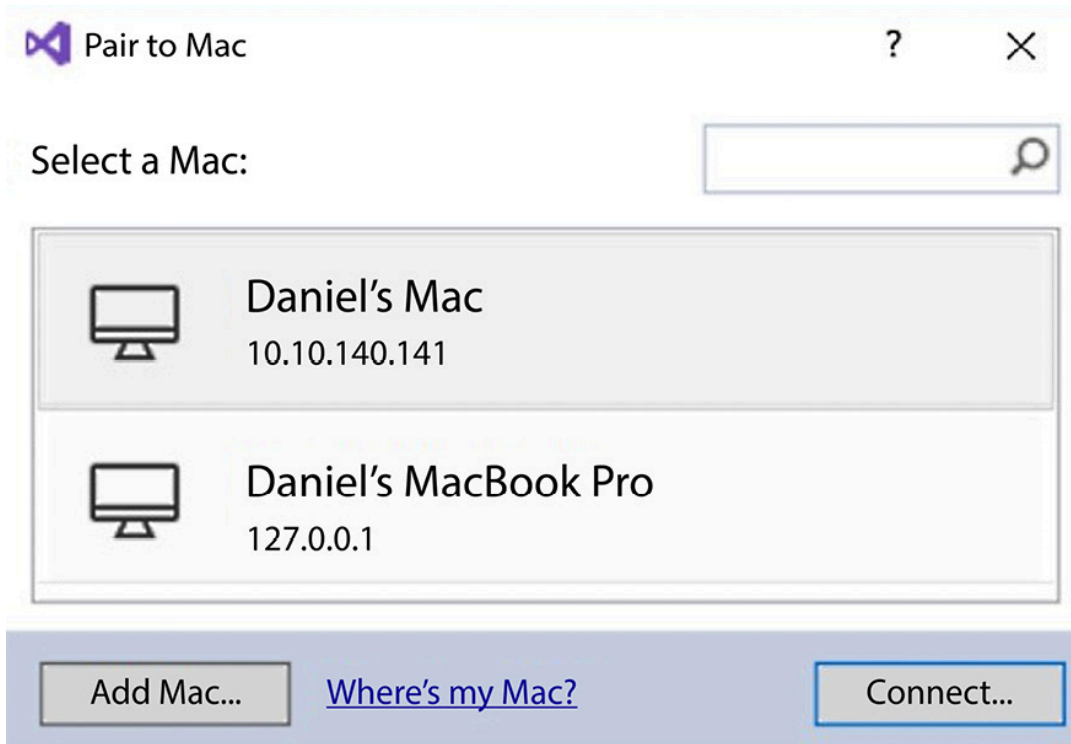


Figure 1.19 – Pair to Mac

If everything that we need is installed on the Mac, then Visual Studio will connect, and we can start building and debugging our iOS app. If Mono is missing on the Mac, a warning will appear. This warning will also give us the option to install it, as shown:

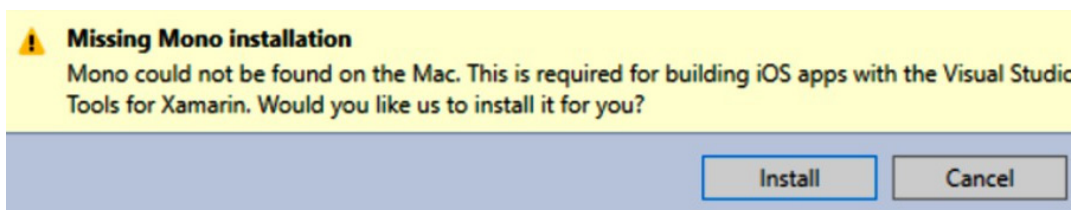


Figure 1.20 – Missing Mono installation dialog

Now that we have our Mac paired, we can get the Android emulator configured.

Configuring an Android emulator and hardware acceleration

If we want a fast Android emulator that works smoothly, we need to enable hardware acceleration. This can be done using either Intel HAXM or Hyper-V. The disadvantage of Intel HAXM is that it can't be

used on machines with an **Advanced Micro Devices (AMD)** processor; we must use a machine with an Intel processor. We can't use Intel HAXM in parallel with Hyper-V.

Because of this, Hyper-V is the preferred way to hardware-accelerate an Android emulator on a Windows machine. To use Hyper-V with our Android emulator, we need to have Windows 11 or Windows 10 with the April 2018 update (or later), and Visual Studio 2017 version 15.8 (or later) installed.

FIND YOUR VERSION OF VISUAL STUDIO

*To determine the version of Visual Studio you are using, when Visual Studio is open, use the **Help | About Visual Studio** menu, and you should be presented with a dialog similar to the following:*

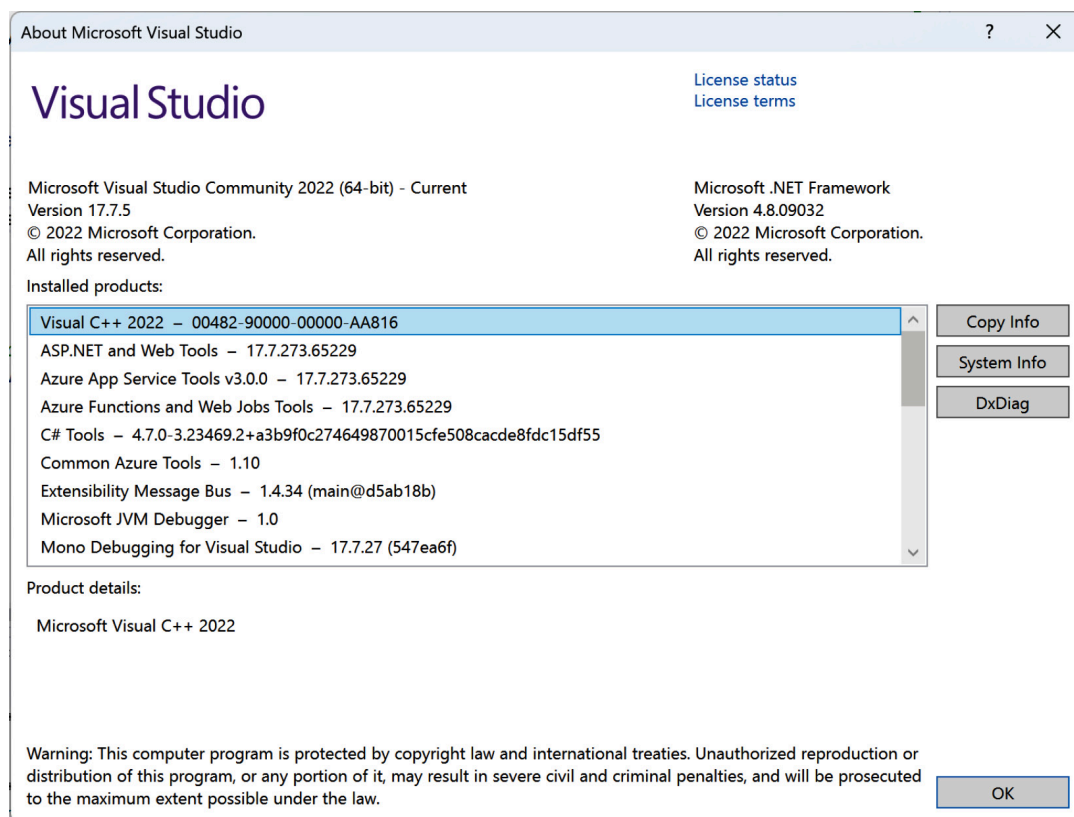


Figure 1.21 – Help | About

To enable Hyper-V, we need to take the following steps:

1. Open the search menu and type in **Turn Windows features on or off**. Click the option that appears to open it, as shown:

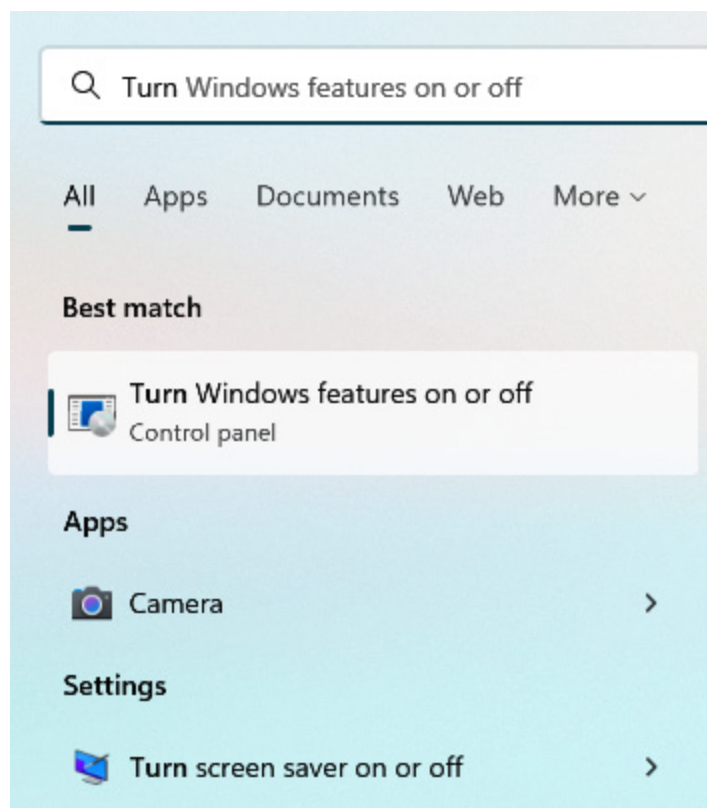


Figure 1.22 – Turn Windows features on or off

2. To enable Hyper-V, select the **Hyper-V** checkbox. Also, expand the **Hyper-V** option and check the **Hyper-V Platform** checkbox. We also need to select the **Windows Hypervisor Platform** checkbox, as shown:

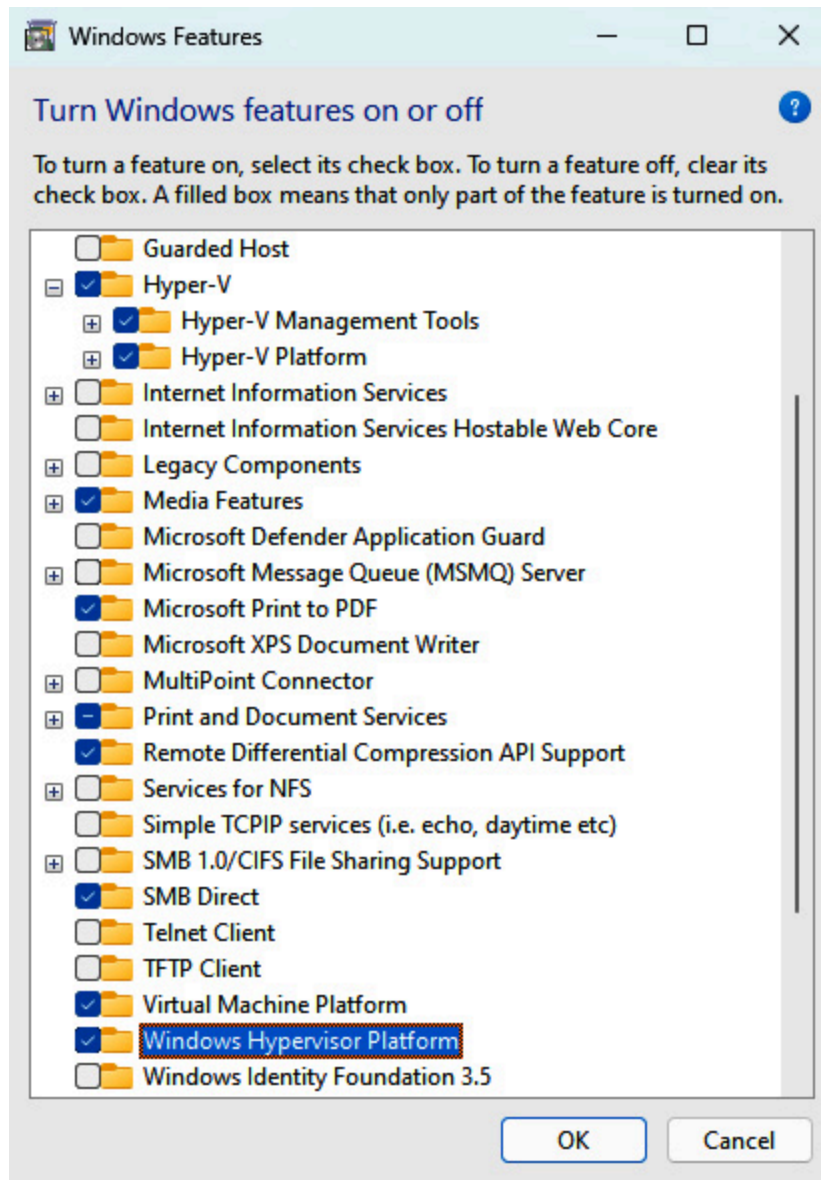


Figure 1.23 – Enabling Hyper-V in Windows Features

3. Restart the machine when Windows prompts you to do so.
4. Because we didn't install an Android emulator during the installation of Visual Studio, we need to install it now. Go to the **Tools** menu in Visual Studio, then click on **Android** and then **Android SDK Manager**.
5. Under **Tools** in **Android SDK Manager**, we can install the emulator by selecting **Android Emulator**, as in the following screenshot. Also, we should ensure that the latest version of **Android SDK Build Tools** is installed:

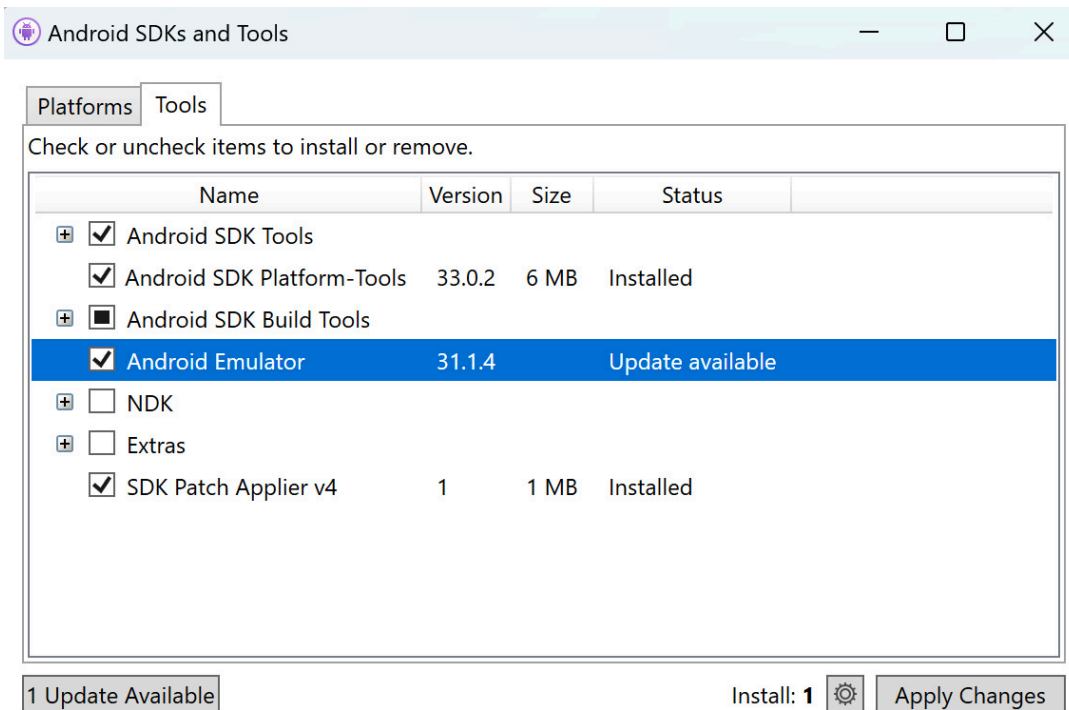


Figure 1.24 – Installing Android Emulator in Android SDK Manager

6. The Android SDK allows multiple emulator images to be installed simultaneously. We can install multiple images if, for example, we want to run our app on different versions of Android. Select emulators with **Google Play** (as in the following screenshot) so that we can use Google Play services in our app, even when we are running it in an emulator.

This is required if, for example, we want to use Google Maps in our app:

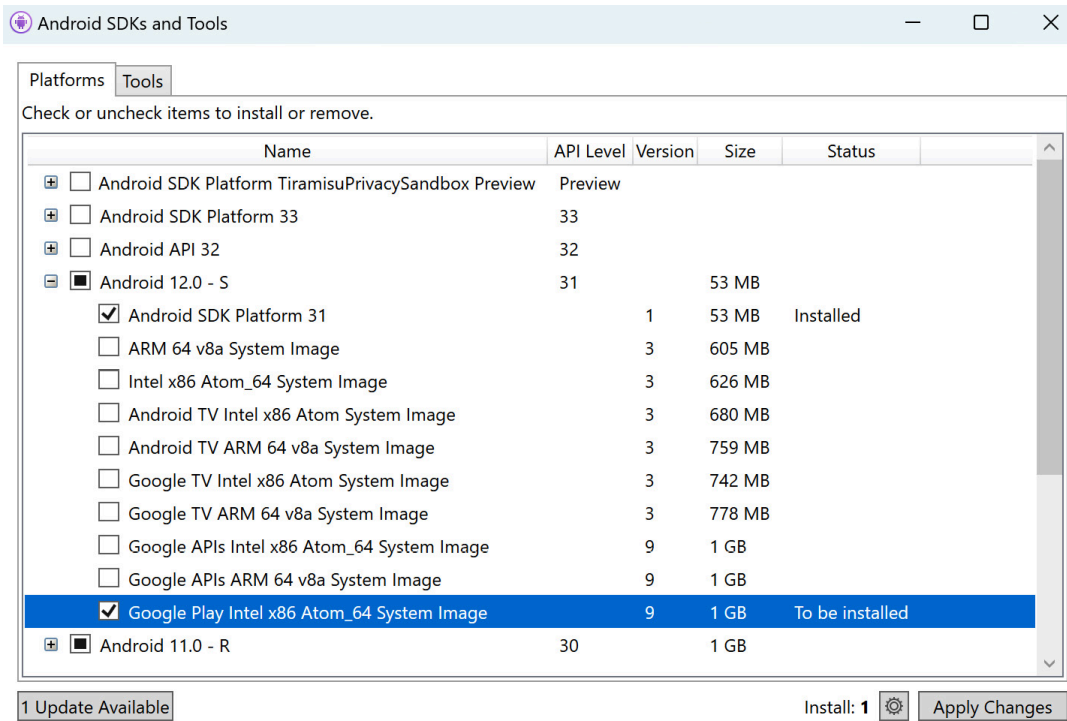


Figure 1.25 – Installing Android Emulator images in Android SDK Manager

7. Be sure to click the **Apply Changes** button to install any components you selected before closing the window.
8. The next step is to create a virtual device to use the emulator image. To create and configure an emulator, go to **Android Device Manager**, which we can open from the **Tools** tab in Visual Studio. From the device manager, we can either start an emulator—if we already have one created—or we can create new emulators, as shown:

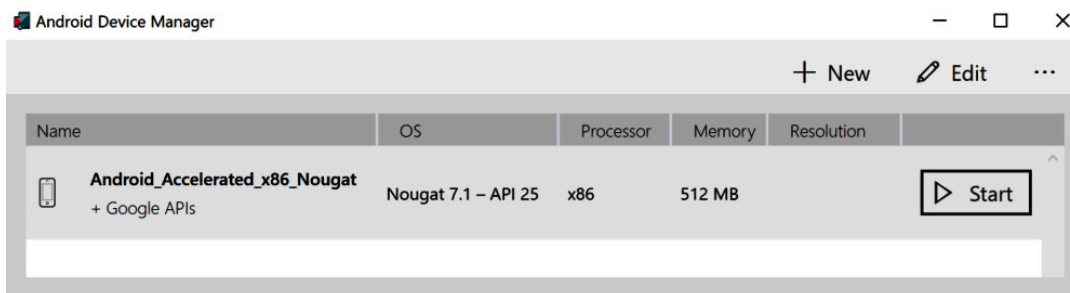


Figure 1.26 – Android Device Manager

If we click on the **New** button, we can create a new emulator with the specifications that we need. The easiest way to create a new emulator

here is to select a base device that matches our needs. These base devices are preconfigured, which is often enough. However, it is possible to edit the properties of the device so that we have an emulator that matches our specific needs.

We must select the correct processor for the emulator to match the processor in our Windows development machine. If it doesn't match, then the emulator will be slower than it needs to be. Select the **x86_64** processor (as in the following screenshot) if you are using Intel or AMD x86-based hardware, or **arm64-v*** if you have an ARM device running Windows.

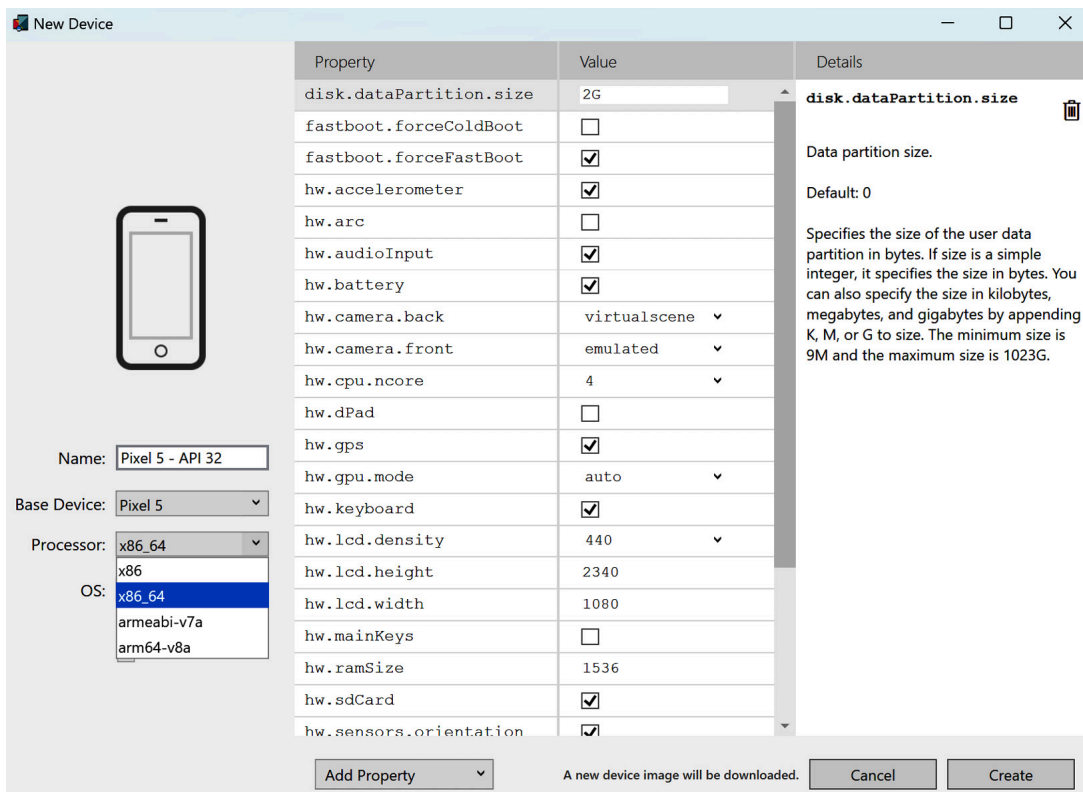


Figure 1.27 – Creating a new device in Android Device Manager

Configuring Developer Mode

If we want to develop desktop apps for Windows, we need to activate Developer Mode on our development machine. To do this, go to **Settings | Privacy & security | For developers**. Then, select **Developer Mode**, as in the following screenshot. This makes it possible for us to sideload and debug apps via Visual Studio:

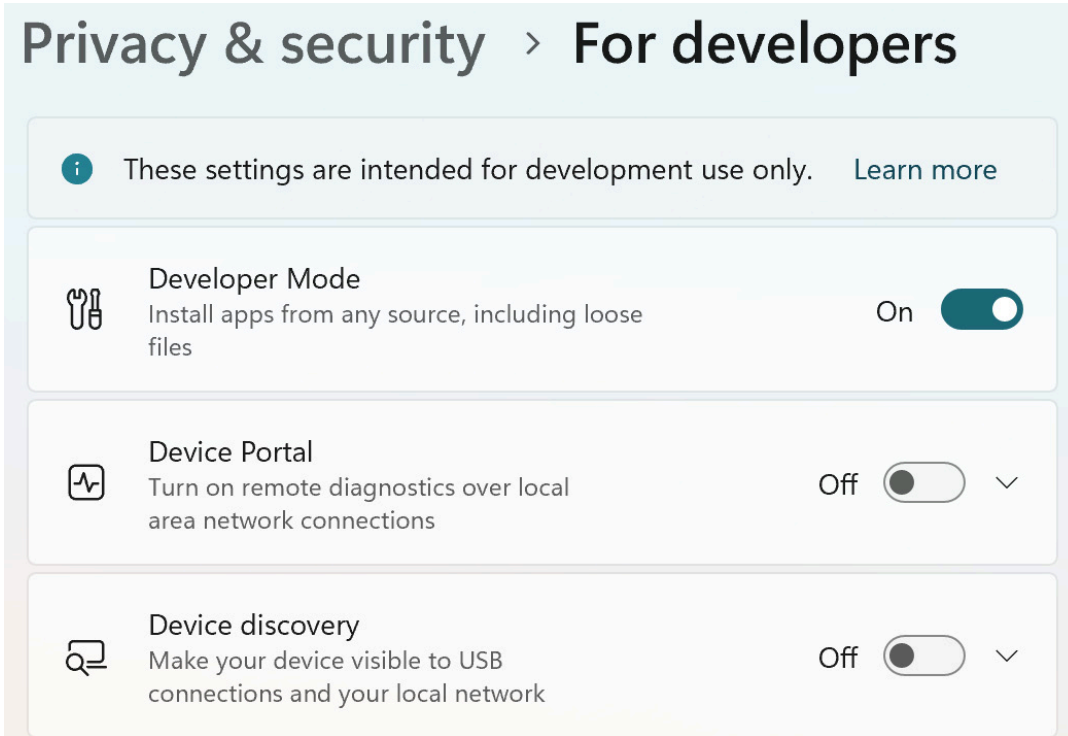


Figure 1.28 – Enabling Developer Mode

At this point, our Windows machine is ready for development. Before we dive into creating our first project, there are a few more optional features that we should review. These will help your development process as you build your apps.

.NET mobile productivity tooling

Xamarin Hot Restart and **Hot Reload** are two tools that increase productivity for .NET MAUI developers. To get even better performance from your Android emulators, you can use the **Windows Subsystem for Android (WSA)**.

Xamarin Hot Restart

Hot Restart is a Visual Studio feature to make developers more productive. It also gives us a way of running and debugging iOS apps on an iPhone without having to use a Mac connected to Visual Studio. Microsoft describes Hot Restart as follows:

Xamarin Hot Restart enables you to quickly test changes to your app during development, including multi-file code edits, resources, and references. It pushes the new changes to the existing app bundle on the debug target which results in a much faster build and deploy cycle.

To use Hot Restart, you need the following:

- Visual Studio 2019 version 16.5
- iTunes (Microsoft Store or 64-bit versions)
- An Apple Developer account and paid Apple Developer Program (<https://developer.apple.com/programs/>) enrollment

Hot Restart can currently only be used with .NET for iOS apps.

Read more about the current state of Hot Restart at

<https://docs.microsoft.com/en-us/xamarin/xamarin-forms/deploy-test/hot-restart>.

Hot Reload

Hot Reload is a runtime technology that allows us to update our running app with the changes we are making in the IDE. There are two major flavors of Hot Reload today: **XAML Hot Reload** and **C# Hot Reload**.

XAML Hot Reload allows us to make changes to our XAML without having to redeploy our app. When we have carried out changes to the XAML, we just save the file, and it updates the page on the simulator/emulator or on a device. XAML Hot Reload is currently supported by all .NET MAUI platforms.

C# Hot Reload allows us to make changes to our code without having to redeploy our app. C# Hot Reload is like *Edit & Continue*; however, you do not have to be in break mode in order to apply the changes to the app. Once you have made changes to your code, you can click the **Hot Reload** button in the toolbar of Visual Studio and Hot Reload will

update the running app. If, for some reason, the changes cannot be applied, Hot Reload will display a dialog either asking you to fix any compilation errors or, in some cases, requiring you to restart the app.

To enable XAML Hot Reload for Visual Studio on Windows, go to **Tools | Options | Xamarin | Hot Reload**.

To enable XAML Hot Reload for Visual Studio on Mac, go to **Visual Studio | Preferences | Tools for Xamarin | XAML Hot Reload**.

C# Hot Reload is only available in Visual Studio for Windows; to enable it, go to **Tools | Options | Debugger | .NET / C++ Hot Reload**.

Windows Subsystem for Android

If you are using Windows 11 in a supported region, you can use WSA as your debugging target instead of the Android emulators. To learn more about WSA and how to set up your machine to use it, visit <https://learn.microsoft.com/en-us/windows/android/wsa/>.

If you want to use WSA to debug your .NET MAUI apps, it will help if you install the WSA Barista Visual Studio extension (<https://marketplace.visualstudio.com/items?itemName=Redth.WindowsSubsystemForAndroidVisualStudioExtension>). This will add the **Windows Subsystem for Android** menu item under **Tools**, which will prompt you to install WSA from the Windows Store, and then automatically configure WSA and set up Visual Studio to use WSA as a device.

Summary

You should now feel a bit more comfortable with what .NET mobile is and how .NET MAUI relates to .NET mobile.

In this chapter, we established a definition of what a native app is and saw how it has a native UI, performance, and API access. We talked about how .NET mobile uses Mono, which is an open source implementation of the .NET Framework, and discussed how, at its core, .NET mobile is a set of bindings to platform-specific APIs. We then looked at how .NET for iOS and .NET for Android work under the hood.

After that, we began to touch on the core topic of this book, which is .NET MAUI. We started with an overview of how platform-agnostic controls are rendered to platform-specific controls and how to use XAML to define a hierarchy of controls to assemble a page. We then spent some time looking at the difference between a .NET MAUI app and a traditional .NET mobile app.

A traditional .NET mobile app uses platform-specific APIs directly, without any abstraction, other than what .NET adds as a platform. .NET MAUI is an API that is built on top of the traditional .NET APIs and allows us to define platform-agnostic GUIs in XAML or in code that is rendered to platform-specific controls. There's more to .NET MAUI than this, but this is what it does at its core.

In the last part of this chapter, we discussed how to set up a development machine on Windows or macOS. Finally, we looked at three optional features that you can use to help improve your development cycle: Hot Restart, Hot Reload, and WSA.

Now, it's time to put our newly acquired knowledge to use! We will start by creating a *to-do* app from the ground up in the next chapter. We will look at concepts such as **Model–View–ViewModel (MVVM)** for a clean separation between business logic and the UI and SQLite.NET to persist data to a local database on our device. We will do this for three platforms at the same time—so, read on!

